HiveOps Backend Architecture & Admin Specifications

This document details the internal logic, data flow, and architectural separation between the **Beekeeper Mobile App** and the **Admin Dashboard**.

1. System Architecture Overview

HiveOps uses a **decoupled architecture** to ensure the heavy lifting (ML and Admin management) doesn't slow down the beekeeper's field experience.

- **Frontend (Beekeeper):** A React/Mobile-first PWA for field operations.
- **Frontend (Admin):** A separate, protected React dashboard for system oversight.
- **Main API (Node.js/Express):** Orchestrates data, authentication, and communication between services.
- **ML Microservice (FastAPI):** Specialized service for processing NDVI, satellite data, and weather for suitability scoring.
- **Database (MongoDB):** Geospatial database storing hive locations, farmer availability, and heatmaps.

2. Core Data Entities (The "What")

A. The User Model (Unified Auth)

There is only one login portal for users.

- **Roles:** `beekeeper` (default) or `admin`.
- **Farmers:** Are **not** users in the system. They are "Resources" or "Entities" managed by admins and discovered by beekeepers.

B. The Farmer Entity (Nearby Discovery)

Collection: `farmers`

Field	Type	Description
`location`	GeoJSON Point	`[lng, lat]` coordinates for spatial queries.
`status`	String	`pending`, `approved`, `rejected`.
`crop_type`	Array	List of flora (e.g., "Sunflower", "Lavender").

`is_public`	Boolean	If true, beekeepers can see them on the map.

C. The GeoTile (ML Cache)

Collection: `geotiles` To prevent expensive ML calls, scores are cached here for 6 hours. If a beekeeper requests a score for a coordinate, the system checks if a `tileKey` (lat_lng) exists before calling the ML service.

3. Key Backend Workflows (The "How")

Workflow: Finding a Relocation Spot

1. **Request:** Beekeeper sends `POST /api/predict` with current coordinates.
2. **ML Inference:** Backend calls FastAPI service to get flowering levels and suitability scores.
3. **Farmer Lookup:** Backend performs a `$near` MongoDB query on the `farmers` collection:
   - *Filter:* Only return farmers where `status === "approved"` .
   - *Radius:* Usually within 10-50km.
4. **Response:** The app displays the score AND a list/map markers of these farmers for potential hive relocation.

Workflow: Offline-First Syncing

1. Beekeeper performs an inspection in a "No Signal" zone.
2. App stores the inspection in a local queue.
3. When signal returns, app calls `POST /api/sync` .
4. Backend processes the array of actions, updates MongoDB, and returns the new `syncVersion` .

4. Admin Dashboard (Separate System)

The Admin Dashboard is hosted at a different URL (e.g., `admin.hiveops.app`). It handles the "Gatekeeping" logic of the ecosystem.

Admin Dashboard API Logic

Feature	Endpoint	Logic / Business Rule
Farmer Vetting	`GET /api/admin/requests`	Fetches all farmer profiles where `status: "pending"` .

Approval	`PATCH /api/admin/farmers/:id`	Changes status to `approved`. This action triggers the farmer's appearance on the Beekeeper's "Discovery" map.
Beekeeper Verification	`PATCH /api/admin/users/:id`	Admins can set `isVerified: true` for beekeepers to allow them access to premium allocation tools.
System Health	`GET /api/admin/telemetry`	Aggregates data: Total Hives, Active Syncs, and ML Service Latency.

Why Separate the Admin System?

- **Security:** Admin routes are protected by a specific middleware (`isAdmin`) that checks the JWT role.
- **No Farmer Login:** By keeping farmers as "Data Entities" rather than "Users," we simplify the security model. The Admin acts as the trusted middleman who enters/verifies farmer data.
- **Performance:** The main HiveOps app doesn't need to load the heavy data-grid libraries required for the Admin Panel.

5. API Response Standards

All APIs return a consistent wrapper to simplify frontend error handling:

```
{
  "success": true,
  "data": {
    "nearby_farmers": [
      { "name": "John's Orchard", "location": [74.0, 18.0], "crop": "Apple" }
    ],
    "prediction": { "score": 88, "trend": "improving" }
  }
}
```

Generated for HiveOps Technical Documentation — April 2026